SentinelHash — File Integrity Monitoring

Cybersecurity 101 | Individual Lab Assignment

Student: Abdul Moeez | Lab date: 5 November 2026

Environment: Ubuntu 26.04.1 LTS (WSL), Python 3.14.4, SHA-256

Part A — Core security concepts

Integrity: Keeping data accurate and unchanged unless an authorized person makes a change. File integrity monitoring checks whether files differ from a trusted earlier version.

Hashing: A one-way process that produces a fixed-length digest from data. Even a small content change normally produces a different SHA-256 digest. A hash is not encryption.

Encryption: Converting readable data into ciphertext using a key, so authorized users can decrypt it and recover the original data.

Digital signatures: Cryptographic proof created with a private key and verified using a corresponding public key. Signatures help verify origin and detect tampering.

Part B — Hash algorithm comparison

Algorithm	Digest	Security assessment
SHA-256	256 bits	Recommended for new file-integrity checks; no practical collision attack is known.
SHA-1	160 bits	Collision attacks are practical; not suitable for new security-sensitive integrity designs.
MD5	128 bits	Collision attacks are practical; unsuitable for security-sensitive integrity verification.

Choice: I would use SHA-256 because it has a much stronger collision-resistance margin than SHA-1 or MD5. A hash comparison detects content differences, but a trusted, protected baseline is still necessary.

Objective

Create a baseline of two lab files, compare subsequent SHA-256 hashes, and report added, modified, and deleted files. All changes were made intentionally within an owned training folder.

Part C — Lab implementation

Monitored folder: /home/moeez/FIM-Lab **Baseline:** /home/moeez/baseline.csv

The lab began with critical.txt and readme.txt. A CSV recorded each absolute file path and its SHA-256 digest. The Python script loaded that baseline, rescanned the directory, and compared the two sets of paths and hashes.

Comparison rule	Finding
Path only in current scan	ADDED
Path only in baseline	DELETED
Same path, different SHA-256	MODIFIED
Same path, matching SHA-256	No content change detected

Observed results

Stage	Baseline / current	Detected events
Clean scan	2 / 2	0
After modifying critical.txt	2 / 2	1 — MODIFIED
After adding new.txt	2 / 3	2 — MODIFIED, ADDED
After deleting readme.txt	2 / 2	3 — MODIFIED, ADDED, DELETED

The baseline remained unchanged throughout the tests. Previously introduced changes therefore continued to appear in later reports. The Python script exported timestamped CSV findings, and each test report was saved separately.

Baseline and clean-scan evidence

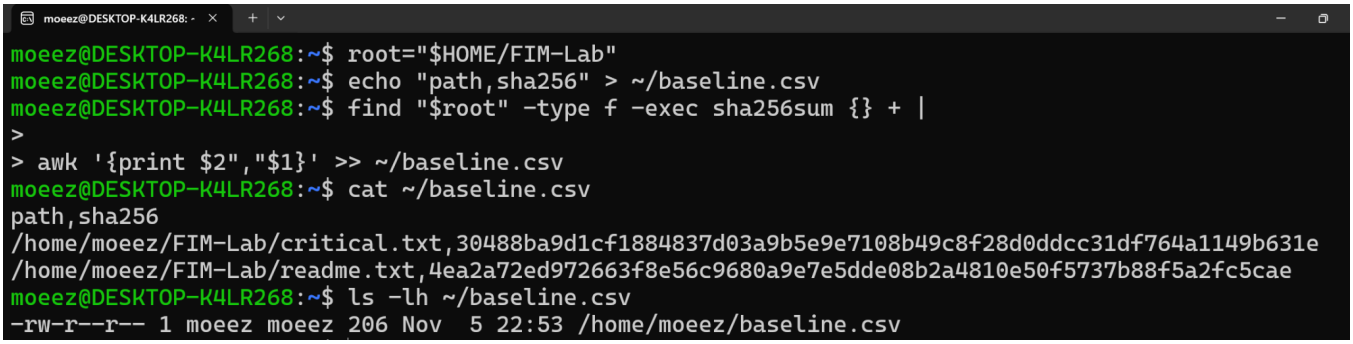


Figure 1. Original baseline CSV with two file paths and SHA-256 hashes.

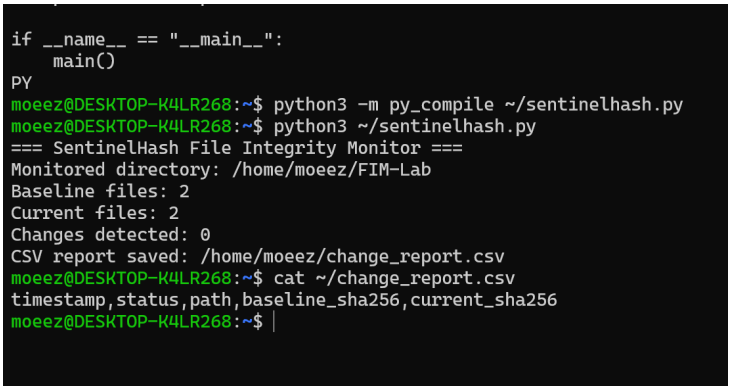


Figure 2. Initial comparison detected zero changes.

Part D — Detection evidence

Test A — Modified file

I appended a line to critical.txt, ran the monitor, and received one MODIFIED finding. The baseline and current SHA-256 values differed.

```
moeez@DESKTOP-K4LR268:~$ cat ~/change_report.csv
timestamp,status,path,baseline_sha256,current_sha256
moeez@DESKTOP-K4LR268:~$ echo "unauthorized configuration change" >> ~/FIM-Lab/critical.txt
moeez@DESKTOP-K4LR268:~$ python3 ~/sentinelhash.py
=== SentinelHash File Integrity Monitor ===
Monitored directory: /home/moeez/FIM-Lab
Baseline files: 2
Current files: 2
Changes detected: 1
[MODIFIED] /home/moeez/FIM-Lab/critical.txt
CSV report saved: /home/moeez/change_report.csv
moeez@DESKTOP-K4LR268:~$ cat ~/change_report.csv
timestamp,status,path,baseline_sha256,current_sha256
2026-11-05T18:05:24.687106+00:00,MODIFIED,/home/moeez/FIM-Lab/critical.txt,30488ba9d1cf1884837d03a9b5e9e7108b49c8f28d0ddcc31df764a1149b631
e,7af6ffcefcf936f4ae2f208fc0baeb43b116b03b740fac16f3c4a66925501ff8
moeez@DESKTOP-K4LR268:~$ cp ~/change_report.csv ~/report_modified.csv
```

Figure 3. Modified file detected and report_modified.csv preserved.

Test B — Added file

I created new.txt and ran the monitor again. The script identified the new path as ADDED while still reporting the earlier modification.

```
moeez@DESKTOP-K4LR268: ~
moeez@DESKTOP-K4LR268:~$ echo "This is a new test file" > ~/FIM-Lab/new.txt
moeez@DESKTOP-K4LR268:~$ python3 ~/sentinelhash.py
=== SentinelHash File Integrity Monitor ===
Monitored directory: /home/moeez/FIM-Lab
Baseline files: 2
Current files: 3
Changes detected: 2
[MODIFIED] /home/moeez/FIM-Lab/critical.txt
[ADDED] /home/moeez/FIM-Lab/new.txt
CSV report saved: /home/moeez/change_report.csv
moeez@DESKTOP-K4LR268:~$ cat ~/change_report.csv
timestamp,status,path,baseline_sha256,current_sha256
2026-11-05T18:10:22.413039+00:00,MODIFIED,/home/moeez/FIM-Lab/critical.txt,30488ba9d1cf1884837d03a9b5e9e7108b49c8f28d0ddcc31df764a1149b631
e,7af6ffcefcf936f4ae2f208fc0baeb43b116b03b740fac16f3c4a66925501ff8
2026-11-05T18:10:22.413039+00:00,ADDED,/home/moeez/FIM-Lab/new.txt,,f454169a780a99a46d03be23c4173e00b63bc1becf3a297cf07f76684dbd3c4e
moeez@DESKTOP-K4LR268:~$ cp ~/change_report.csv ~/report_added.csv
```

Figure 4. Added file detected and report_added.csv preserved.

Part D — Final detection and analysis

Test C — Deleted file

I deleted readme.txt and ran a final comparison. The monitor reported three events: MODIFIED for critical.txt, ADDED for new.txt, and DELETED for readme.txt.

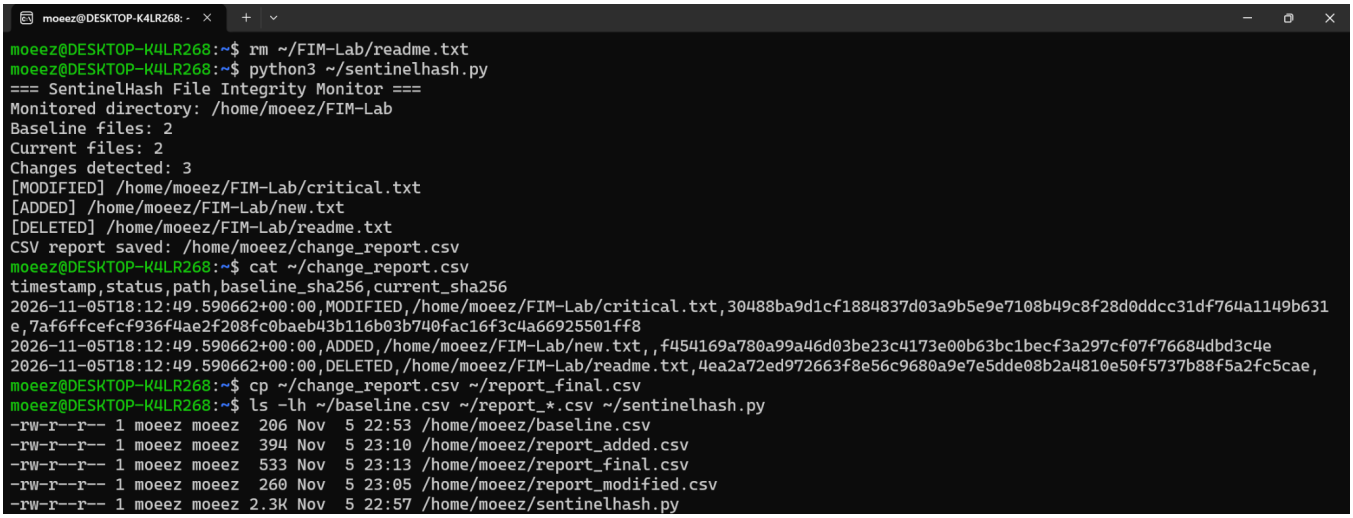


Figure 5. Final scan shows all three event types and the saved CSV reports.

Part E — SOC analyst finding

Finding: The file-integrity monitor identified three deviations from the trusted baseline in the controlled lab: one modified configuration file, one newly added text file, and one deleted text file. These were authorized test actions, so they are not evidence of malicious activity. In a real environment, an analyst would validate the change owner, timing, and maintenance records before escalating.

Risk: Unexpected modifications to configuration files can affect system integrity; unexplained additions or deletions may indicate unauthorized activity or operational mistakes.

Recommended improvement: Protect the baseline in a restricted or centrally managed location and restrict write access. This reduces the chance that an attacker can alter both monitored files and their reference hashes.

Conclusion and limitations

SentinelHash detected the three required types of file changes using SHA-256 comparisons and exported CSV reports. It is a teaching prototype, not a real-time endpoint agent. It does not establish who made a change, whether the change was malicious, or whether an attacker has altered the baseline. The exercise was conducted on Ubuntu under WSL rather than Kali Linux; the SHA-256 comparison workflow is equivalent for this lab.